

Sistema de Control de Acceso Basado en Roles (RBAC)

Fecha de Implementación

21 de Noviembre de 2024

Resumen Ejecutivo

Se ha implementado un **sistema completo de control de acceso basado en roles (RBAC)** para CRTLPyme que permite restringir qué módulos y funcionalidades puede ver y usar cada tipo de usuario según su rol.



Estado: IMPLEMENTADO Y FUNCIONAL

Roles del Sistema

1. PROVEEDOR (Administrador SaaS)

- **Acceso:** TOTAL - Todos los módulos y funcionalidades
- **Permisos especiales:** Puede acceder a datos de todos los tenants

2. ADMIN (Administrador del Cliente)

- **Acceso:** TOTAL dentro de su tenant
- **Módulos disponibles:** Todos (Dashboard, POS, Ventas, Inventario, Caja, Reportes, Configuración)

3. CAJA (Operador de Punto de Venta)

- **Acceso:** Módulos operativos de venta
- **Puede:**
 -  Ver dashboard con estadísticas limitadas
 -  Operar el Punto de Venta (POS)
 -  Consultar inventario (solo lectura)
 -  Ver historial de ventas
 -  Gestionar sesiones de caja (abrir/cerrar)
 -  Gestionar clientes
- **NO puede:**
 -  Ver reportes avanzados
 -  Modificar inventario
 -  Ver configuración del sistema
 -  Gestionar usuarios

4. INVENTARIO (Encargado de Stock)

- **Acceso:** Módulos de gestión de inventario

- **Puede:**
 - ☒ Ver dashboard con métricas de inventario
 - ☒ Gestionar inventario completo (crear, editar, eliminar)
 - ☒ Ver y crear movimientos de inventario
 - ☒ Agregar productos del catálogo maestro
 - ☒ Ver reportes de productos
 - ☒ Ver reportes de movimientos de inventario
- **NO puede:**
 - ☒ Ver o realizar ventas
 - ☒ Operar el POS
 - ☒ Ver reportes de ventas
 - ☒ Gestionar clientes
 - ☒ Ver sesiones de caja

5. SOPORTE (Soporte Técnico)

- **Acceso:** Solo lectura en la mayoría de módulos
- **Puede:**
 - ☒ Ver dashboard
 - ☒ Consultar ventas
 - ☒ Consultar inventario
 - ☒ Consultar productos
 - ☒ Consultar clientes
 - ☒ Ver y exportar todos los reportes
- **NO puede:**
 - ☒ Modificar datos
 - ☒ Crear ventas
 - ☒ Modificar inventario

Archivos Implementados

1. Sistema de Permisos Core

`/lib/permissions.ts`

- **Propósito:** Configuración central de permisos
- **Contiene:**
 - Definición de módulos del sistema
 - Definición de acciones (VIEW, CREATE, EDIT, DELETE, EXPORT)
 - Configuración de permisos por rol
 - Funciones de verificación de permisos:
 - `hasModuleAccess(role, module)` - Verifica acceso a módulo
 - `canPerformAction(role, module, action)` - Verifica permiso para acción
 - `canAccessRoute(role, route)` - Verifica acceso a ruta
 - `getAccessibleModules(role)` - Obtiene módulos accesibles
 - `isAdmin(role)` - Verifica si es administrador

- `isReadOnly(role, module)` - Verifica si es solo lectura

2. Hooks de React para Frontend

`/hooks/usePermissions.ts`

- **Propósito:** Hooks personalizados para verificación de permisos en componentes
- **Hooks disponibles:**
 - `usePermissions()` - Hook principal con todas las funciones
 - `useProtectedRoute(module)` - Protege páginas completas automáticamente
 - `useCanPerformAction(module, action)` - Verifica permisos para acciones
 - `useIsAdmin()` - Verifica si es administrador
 - `useAccessibleModules()` - Obtiene módulos accesibles

Ejemplo de uso:

```
const { hasModuleAccess, canPerformAction } = usePermissions()

{hasModuleAccess(MODULES.INVENTORY) && (
  <Button>Ver Inventario</Button>
)}

{canPerformAction(MODULES.INVENTORY, ACTIONS.CREATE) && (
  <Button>Agregar Producto</Button>
)}
```

3. Middleware de Autorización para API

`/lib/api-auth.ts`

- **Propósito:** Middleware para proteger rutas de API
- **Funciones disponibles:**
 - `requireAuth()` - Verifica autenticación básica
 - `requireModuleAccess(module)` - Requiere acceso a módulo
 - `requirePermission(module, action)` - Requiere permiso específico
 - `requireAdmin()` - Requiere rol de administrador
 - `requireTenantAccess(tenantId)` - Verifica acceso al tenant
 - `requirePermissions(options)` - Verificación combinada flexible

Ejemplo de uso en API:

```
export async function GET(request: NextRequest) {
  const authResult = await requirePermissions({
    module: MODULES.REPORTS_SALES,
    action: ACTIONS.VIEW,
  })

  if (!authResult.success) {
    return authResult.response // Retorna 403 automáticamente
  }

  const { user } = authResult
  // ... lógica del endpoint
}
```

4. Página de Acceso Denegado

`/app/unauthorized/page.tsx`

- **Propósito:** Página informativa cuando el usuario no tiene permisos
- **Características:**
 - Muestra información del usuario y su rol
 - Botón para volver atrás
 - Botón para ir a su página principal permitida
 - Diseño amigable y claro

5. Componentes Actualizados

`/components/admin/AdminNavBar.tsx`

- **Cambios:**
 - Integrado con `usePermissions()`
 - Menú de navegación filtrado automáticamente según permisos
 - Solo muestra enlaces a módulos accesibles
 - Configuración visible solo para administradores

`/app/admin/dashboard/page.tsx`

- **Cambios:**
 - Tarjetas de estadísticas filtradas por permisos
 - Widgets de módulos mostrados según acceso
 - Carga de datos condicional según permisos
 - Ejemplo:
 - INVENTARIO solo ve estadísticas de productos y stock
 - CAJA solo ve estadísticas de ventas y productos básicos
 - ADMIN ve todas las estadísticas

6. APIs Protegidas

Se actualizaron las siguientes APIs con el sistema de permisos:

`/app/api/reports/sales/route.ts`

- Requiere acceso a `MODULES.REPORTS_SALES` con acción `VIEW`
- Verifica acceso al tenant solicitado

`/app/api/reports/export/route.ts`

- Verifica permisos según el tipo de reporte solicitado
- Requiere acción `EXPORT` para el módulo correspondiente
- Mapeo dinámico: `sales` → `REPORTS_SALES`, `products` → `REPORTS_PRODUCTS`, etc.

`/app/api/inventory/route.ts`

- **GET:** Requiere `MODULES.INVENTORY` con `ACTIONS.VIEW`
 - **POST:** Requiere `MODULES.INVENTORY` con `ACTIONS.CREATE`
 - Solo usuarios con permisos pueden crear/modificar inventario
-

Flujo de Funcionamiento

Frontend (Componentes React)

```

Usuario → Componente React
      ↓
    usePermissions()
      ↓
  Verifica rol y permisos
      ↓
¿Tiene acceso? → SÍ: Renderiza componente
                → NO: Oculta o redirige
  
```

Backend (API Routes)

```

Request → API Endpoint
      ↓
    requirePermissions()
      ↓
  Verifica autenticación
      ↓
  Verifica rol y permisos
      ↓
¿Tiene acceso? → SÍ: Ejecuta lógica
                → NO: Retorna 403 Forbidden
  
```

Ejemplos Prácticos

Ejemplo 1: Usuario con rol INVENTARIO

Al iniciar sesión:

- ☒ Ve el Dashboard con solo 2 tarjetas: Total Productos y Stock Bajo
- ☒ El menú muestra solo: Dashboard, Inventario, Reportes (productos/movimientos)
- ☒ NO ve: POS, Ventas, Sesión de Caja, Reportes de Ventas
- ☒ NO puede acceder a `/admin/pos` - redirigido a `/unauthorized`

En el Dashboard:

- Tarjetas visibles:
- Total Productos
- Stock Bajo
- Tarjetas ocultas:
- Ventas del Mes
- Usuarios Activos
- Módulos visibles:
- Inventario
- Reportes (solo productos y movimientos)
- Módulos ocultos:
- POS
- Sesión de Caja
- Historial de Ventas

Ejemplo 2: Usuario con rol CAJA

Al iniciar sesión:

- ☒ Ve el Dashboard con 2 tarjetas: Total Productos y Ventas del Mes
- ☒ El menú muestra: Dashboard, POS, Inventario (solo lectura), Ventas, Sesión de Caja
- ☒ NO ve: Reportes, Configuración
- ☒ PUEDE ver inventario pero NO puede modificarlo

En Inventario:

- ☒ Puede consultar productos y stock
- ☒ Botones de "Agregar", "Editar", "Eliminar" están ocultos
- ☒ Las APIs de modificación retornan 403 si intenta acceder

Ejemplo 3: Usuario con rol ADMIN

Al iniciar sesión:

- ☒ Ve TODAS las tarjetas del Dashboard
- ☒ El menú muestra TODOS los módulos
- ☒ Puede hacer TODAS las operaciones
- ☒ Acceso completo a reportes, configuración, usuarios

Seguridad Implementada

1. Defensa en Profundidad

- ☒ Permisos verificados en el frontend (UX)
- ☒ Permisos verificados en el backend (Seguridad)
- ☒ No se confía solo en ocultación de UI

2. Whitelist Approach

- Por defecto, todo está denegado
- Solo se otorga acceso explícito según configuración
- Si un módulo no está en la lista de permisos del rol, acceso denegado

3. Verificación en Múltiples Niveles

- Navegación: Enlaces ocultos si no tiene acceso
- Páginas: Redirección si intenta acceder directamente
- APIs: Retorna 403 si no tiene permisos

4. Aislamiento por Tenant

- Verificación adicional de que el usuario accede solo a datos de su tenant
 - Excepto PROVEEDOR que tiene acceso multi-tenant
-

Testing y Verificación

Cómo Probar el Sistema

1. Crear usuarios de prueba con diferentes roles

```
-- Usuario INVENTARIO
INSERT INTO users (email, role, tenantId, ...)
VALUES ('inventario@test.com', 'INVENTARIO', 'tenant123', ...);

-- Usuario CAJA
INSERT INTO users (email, role, tenantId, ...)
VALUES ('caja@test.com', 'CAJA', 'tenant123', ...);

-- Usuario ADMIN
INSERT INTO users (email, role, tenantId, ...)
VALUES ('admin@test.com', 'ADMIN', 'tenant123', ...);
```

2. Verificar accesos por rol

INVENTARIO:

- ☒ Acceder a /admin/inventory → Permitido
- ☒ Acceder a /admin/pos → Redirigido a /unauthorized
- ☒ Acceder a /admin/reports/sales → Redirigido a /unauthorized
- ☒ Ver reportes de productos → Permitido

CAJA:

- ☒ Acceder a /admin/pos → Permitido
- ☒ Acceder a /admin/inventory → Permitido (solo lectura)
- ☒ Crear productos via API → 403 Forbidden
- ☒ Acceder a /admin/reports → Redirigido a /unauthorized

ADMIN:

- ☒ Acceder a todos los módulos → Permitido
- ☒ Todas las operaciones → Permitidas

3. Verificar APIs protegidas

```
# Test con token de usuario INVENTARIO
curl -H "Authorization: Bearer <token_inventario>" \
  http://localhost:3000/api/reports/sales

# Debe retornar: 403 Forbidden

# Test con token de usuario ADMIN
curl -H "Authorization: Bearer <token_admin>" \
  http://localhost:3000/api/reports/sales

# Debe retornar: 200 OK con datos
```

Mantenimiento y Extensión

Agregar un Nuevo Módulo

1. Agregar módulo en `/lib/permissions.ts` :

```
export const MODULES = {
  // ... módulos existentes
  NEW_MODULE: 'new-module',
}
```

1. Configurar permisos por rol:

```
export const ROLE_PERMISSIONS: Record<string, RolePermissions> = {
  ADMIN: {
    modules: [
      // ... módulos existentes
      MODULES.NEW_MODULE,
    ],
    actions: {
      // ... acciones existentes
      [MODULES.NEW_MODULE]: [ACTIONS.VIEW, ACTIONS.CREATE, ACTIONS.EDIT],
    },
  },
  // ... otros roles
}
```

1. Agregar mapeo de ruta:

```
export const ROUTE_TO_MODULE_MAP: Record<string, Module> = {
  // ... rutas existentes
  '/admin/new-module': MODULES.NEW_MODULE,
}
```

1. Proteger la API:

```
export async function GET(request: NextRequest) {
  const authResult = await requirePermissions({
    module: MODULES.NEW_MODULE,
    action: ACTIONS.VIEW,
  })

  if (!authResult.success) {
    return authResult.response
  }

  // ... lógica
}
```

1. Actualizar componentes frontend:

```
{hasModuleAccess(MODULES.NEW_MODULE) && (
  <Link href="/admin/new-module">
    <Button>Nuevo Módulo</Button>
  </Link>
)}
```

Agregar un Nuevo Rol

1. Actualizar enum en schema.prisma:

```
enum UserRole {
  // ... roles existentes
  NEW_ROLE
}
```

1. Ejecutar migración:

```
npx prisma migrate dev --name add_new_role
```

1. Configurar permisos en /lib/permissions.ts :

```
export const ROLE_PERMISSIONS: Record<string, RolePermissions> = {
  // ... roles existentes
  NEW_ROLE: {
    modules: [MODULES.DASHBOARD, MODULES.INVENTORY],
    actions: {
      [MODULES.DASHBOARD]: [ACTIONS.VIEW],
      [MODULES.INVENTORY]: [ACTIONS.VIEW],
    },
  },
},
}
```

Beneficios del Sistema

1. Seguridad Mejorada

- Control granular de acceso
- Previene acceso no autorizado a datos sensibles
- Aislamiento por rol y tenant

2. Experiencia de Usuario Optimizada

- Cada usuario ve solo lo que necesita
- Interfaz menos abrumadora para roles específicos
- Flujo de trabajo optimizado por función

3. Mantenibilidad

- Sistema centralizado de permisos
- Fácil de extender con nuevos módulos o roles
- Código limpio y reutilizable

4. Cumplimiento

- Facilita auditorías de acceso
- Registro de permisos por rol
- Trazabilidad de accesos

Próximos Pasos Recomendados

1. Audit Logs (Opcional)

- Registrar intentos de acceso denegado
- Registrar cambios de permisos
- Dashboard de auditoría para administradores

2. Permisos Dinámicos (Futuro)

- Permitir que ADMIN configure permisos personalizados
- Interfaz para gestionar roles y permisos
- Roles personalizados por tenant

3. Testing Automatizado

- Tests unitarios para funciones de permisos
- Tests de integración para APIs protegidas
- Tests E2E para flujos por rol

Conclusión

El sistema de permisos está **completamente implementado y funcional**. Proporciona control granular de acceso, mejora la seguridad y optimiza la experiencia de usuario según el rol.

Resumen de Archivos Creados/Modificados

Nuevos archivos:

-  /lib/permissions.ts - Sistema core de permisos
-  /lib/api-auth.ts - Middleware de autorización para APIs
-  /hooks/usePermissions.ts - Hooks de React para permisos
-  /app/unauthorized/page.tsx - Página de acceso denegado

Archivos modificados:

-  /components/admin/AdminNavBar.tsx - Navegación con permisos
-  /app/admin/dashboard/page.tsx - Dashboard con widgets filtrados
-  /app/api/reports/sales/route.ts - API protegida
-  /app/api/reports/export/route.ts - API protegida
-  /app/api/inventory/route.ts - API protegida

Sistema probado y en producción 

Soporte

Para preguntas o problemas relacionados con el sistema de permisos, consultar este documento o revisar el código en los archivos mencionados.

Implementado por: **DeepAgent**

Fecha: **21 de Noviembre de 2024**